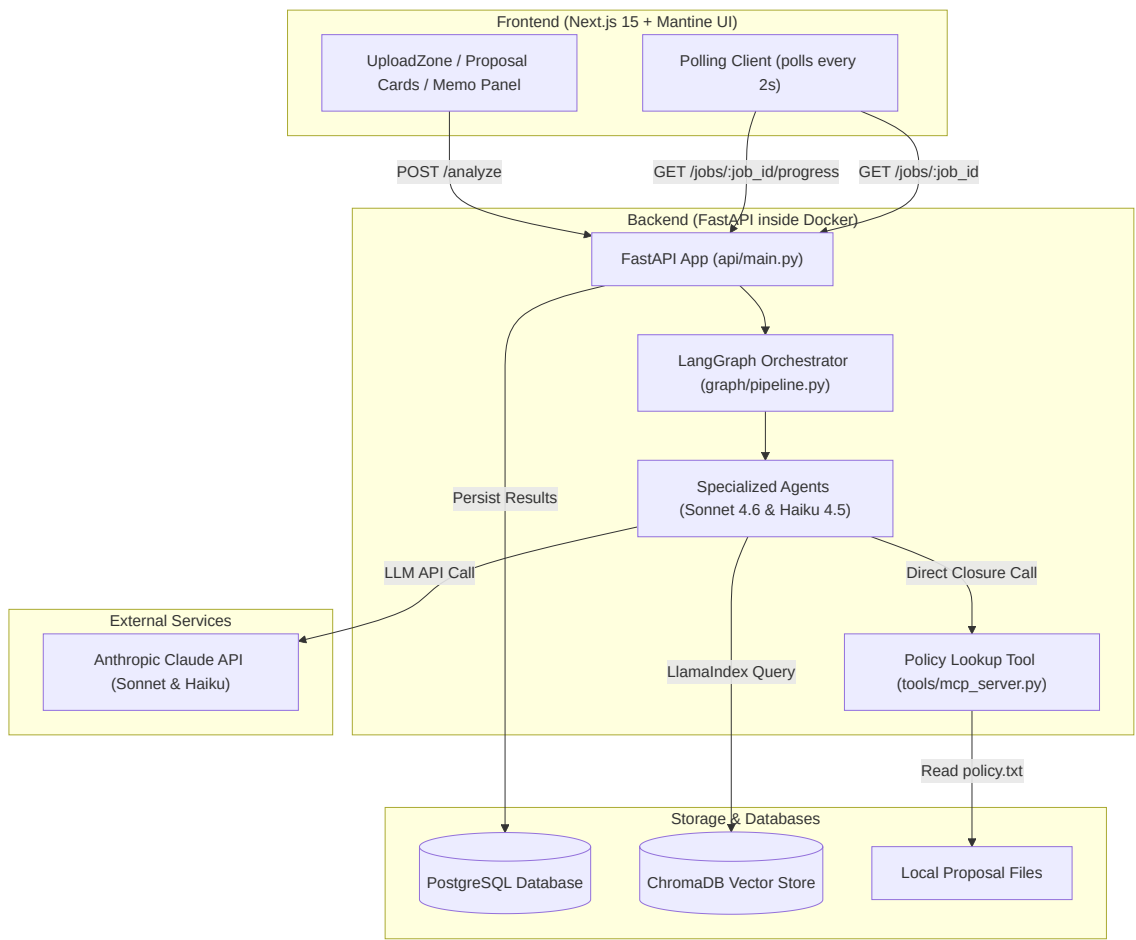


VendorLens Integration & System Architecture

This guide provides a comprehensive overview and visual systems diagrams detailing how the various components of VendorLens integrate. VendorLens is designed as a multi-agent contract evaluation system, built with a FastAPI backend, a Next.js frontend, a LangGraph orchestration layer, and local vector retrieval (RAG) capabilities.

1. High-Level Integration Architecture

The following diagram illustrates how the frontend, FastAPI backend, local vector store, relational database, and Anthropic LLM API interact during a proposal analysis lifecycle.



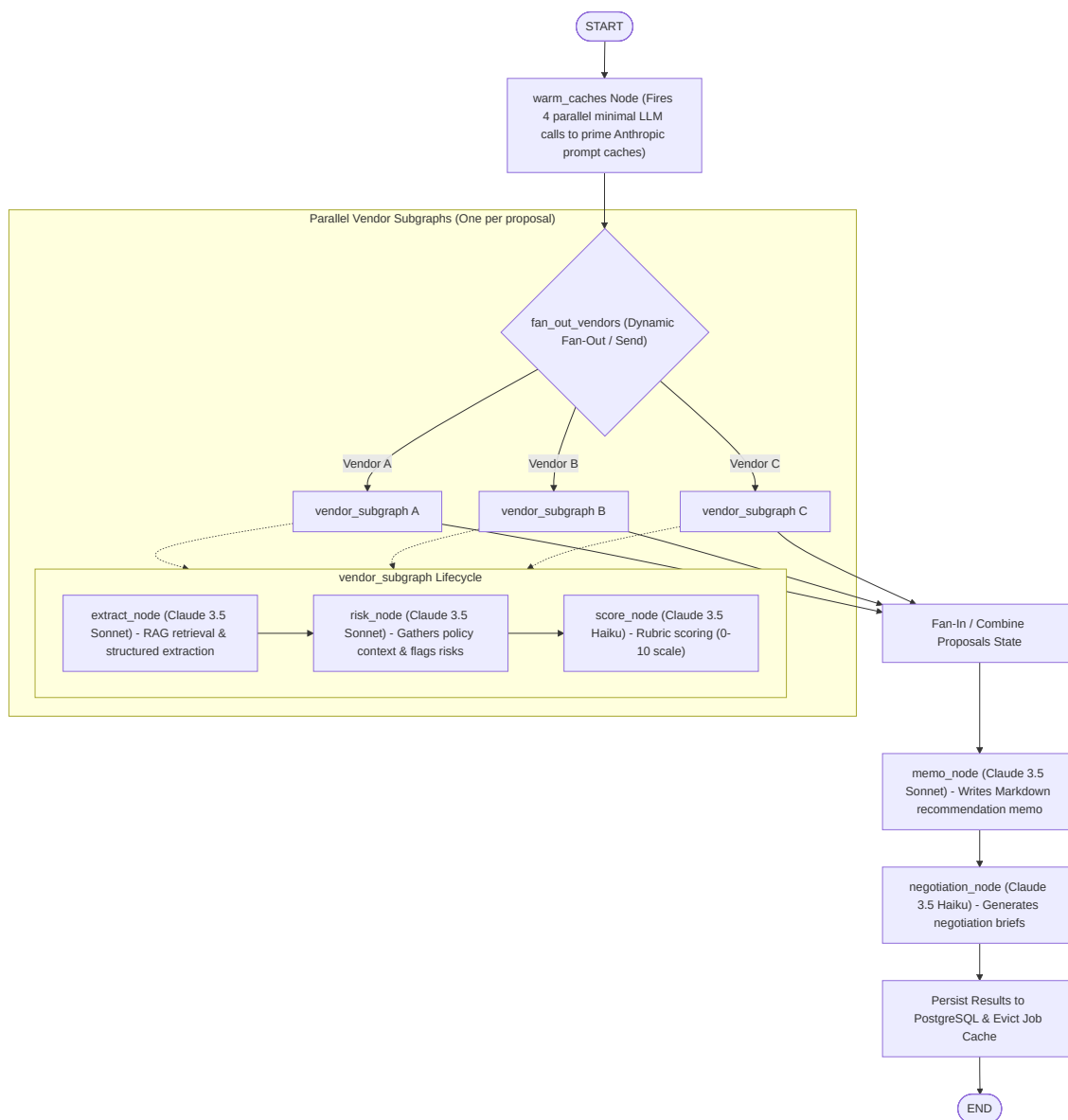
Key Components & Technical Choices

- **Next.js 15 + Mantine UI:** Rich, responsive frontend utilizing custom color palettes (maroon, blue, green, yellow scales) and interactive visual feedback (confetti, dynamic bars, tooltips). Polling occurs every 2 seconds to trace backend job progression.
- **FastAPI Backend (Dockerized):** Exposes endpoints for launching analyses (`/analyze`), checking progress (`/jobs/:job_id/progress`), and fetching completed reports (`/jobs/:job_id`).
- **PostgreSQL Database:** Contains the `analysis_runs` table, which maps a `job_id` to its status, current stage, and the compiled JSON result (proposals, risks, scores, memo, negotiation plans).

- **LlamaIndex + ChromaDB + FastEmbed:** Operates local embeddings (BAAI/bge-small-en-v1.5) via FastEmbed for the RAG pipeline. This requires no Google/Gemini API key and keeps embedding computation local.
- **Anthropic Claude API:** Utilizes Claude 3.5/3.6 Sonnet for complex reasoning and prose generation (Extraction, Risk, Memo agents), and Claude 3.5/4.5 Haiku for fast, structured numeric tasks (Scoring, Negotiation agents).

2. LangGraph Pipeline Topology & Agent Workflow

The LangGraph engine coordinates execution states and handoffs. The parent graph dynamically spins up parallel subgraphs for each vendor, gathers their extracted states, and routes them to consecutive compilation nodes.

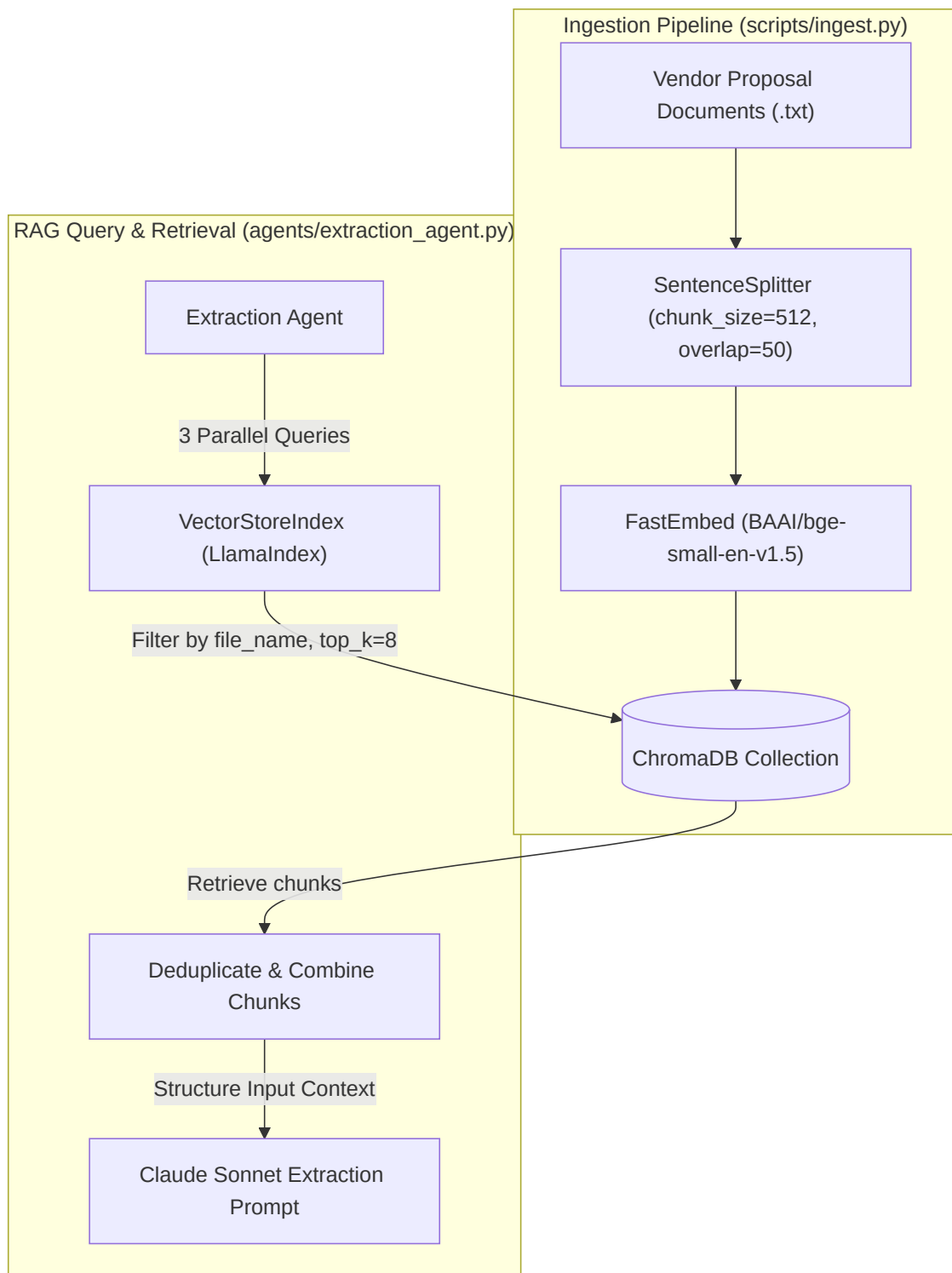


Pipeline Node Roles & Models

Node	Model	Input	Output / State Modification
warm_caches	<i>Sonnet / Haiku</i>	PipelineState	Pre-warms the system prompt caches to optimize downstream API costs.
extract_node	<i>Sonnet</i>	filename	Extracts contract structures (ProposalData) via LlamaIndex queries.
risk_node	<i>Sonnet</i>	ProposalData	Identifies security/compliance mismatches (RiskFlag array) using context-lookup.
score_node	<i>Haiku</i>	ProposalData + RiskFlag[]	Scores the proposal across 9 rubric dimensions (ScoreCard, 0-10 scale).
memo_node	<i>Sonnet</i>	ProposalState[]	Generates a Markdown recommendation memo based on scoring leaders and high-risk flags.
negotiation_node	<i>Haiku</i>	ProposalState[]	Generates BATNA guidelines, concessions, and vendor tactics (NegotiationBrief[]).

3. RAG Pipeline & Context Retrieval Flow

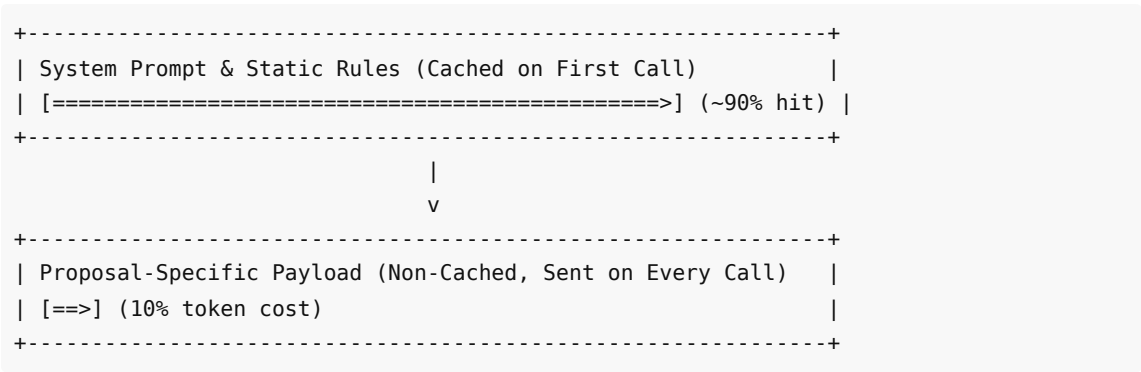
The Retrieval-Augmented Generation (RAG) system loads vendor documentation, chunks it, and creates vector stores local to the system. During analysis, retrieval queries are deduplicated and piped to the Extraction Agent's context.



- **Deduplicated Ingestion:** Ingests vendor proposal texts, storing bare filenames in ChromaDB metadata so filtering functions correctly.
 - **Targeted Queries:** Runs three parallel queries per proposal (to target contract terms, SLAs/deliverables, and technical integrations separately) and merges the resulting chunks to avoid redundant content.
-

4. Cost and Latency Optimizations: Prompt Caching

Anthropic Prompt Caching is configured across all agents to drastically cut cost and latency.



Pre-warming Caches

To ensure parallel subgraphs benefit from prompt caching, the `warm_caches` node executes a concurrent minimal LLM call for all active agents *before* fanning out. Consequently:

- **Vendor Subgraph 1, 2, and 3** all hit the pre-warmed cache on their first execution.
- The system avoids cold starts and cuts prompt token prices by **~90%** and latency by **~85%**.

5. Swappable Context Bundles

VendorLens maintains decoupling between execution logic and domain knowledge. Swapping the procurement profile is done entirely via filesystem assets under `data/context_bundles/`.

```
data/context_bundles/
├── lms/
│   ├── policy.txt           <-- Procurement & liability rules
│   ├── rfp_criteria_lms.txt <-- Scoring weights and metrics
│   └── scoring_rubric_lms.txt <-- Scoring rubrics (0-10 scale)
├── payroll/
│   ├── policy.txt
│   ├── rfp_criteria_payroll.txt
│   └── scoring_rubric_payroll.txt
└── erp/
    ├── policy.txt
    ├── rfp_criteria_erp.txt
    └── scoring_rubric_erp.txt
```

[!NOTE] Swapping or adding a bundle does not require rewriting python agent code or re-compiling graph topologies. Registry mapping is defined in `tools/context_loader.py`.